

Documentación

Lenguaje de Programación JAVA
Universidad Tecnológica Nacional
C.3KE05-2024

Nombre	Legajo	Correo
Colman, Gerónimo	51358	gerocolman82@gmail.com
Munné, Facundo	50361	fnmunne@gmail.com

Índice

Índice.....	1
Diagrama de Clase.....	2
Diagrama de Datos.....	3
Caso de Uso Resumen Reestructurado.....	4
Casos de Uso de Usuario.....	5
CUU 01 - Pedir turno.....	5
CUU 02 - Recepcionar turno.....	6
Capturas de Pantallas.....	7
Login.....	7
Error.....	7
Pantallas de Cliente.....	8
Pantallas de Profesional.....	11
Pantallas de Administrador.....	14
Fragmento de Código.....	17

Diagrama de Clase

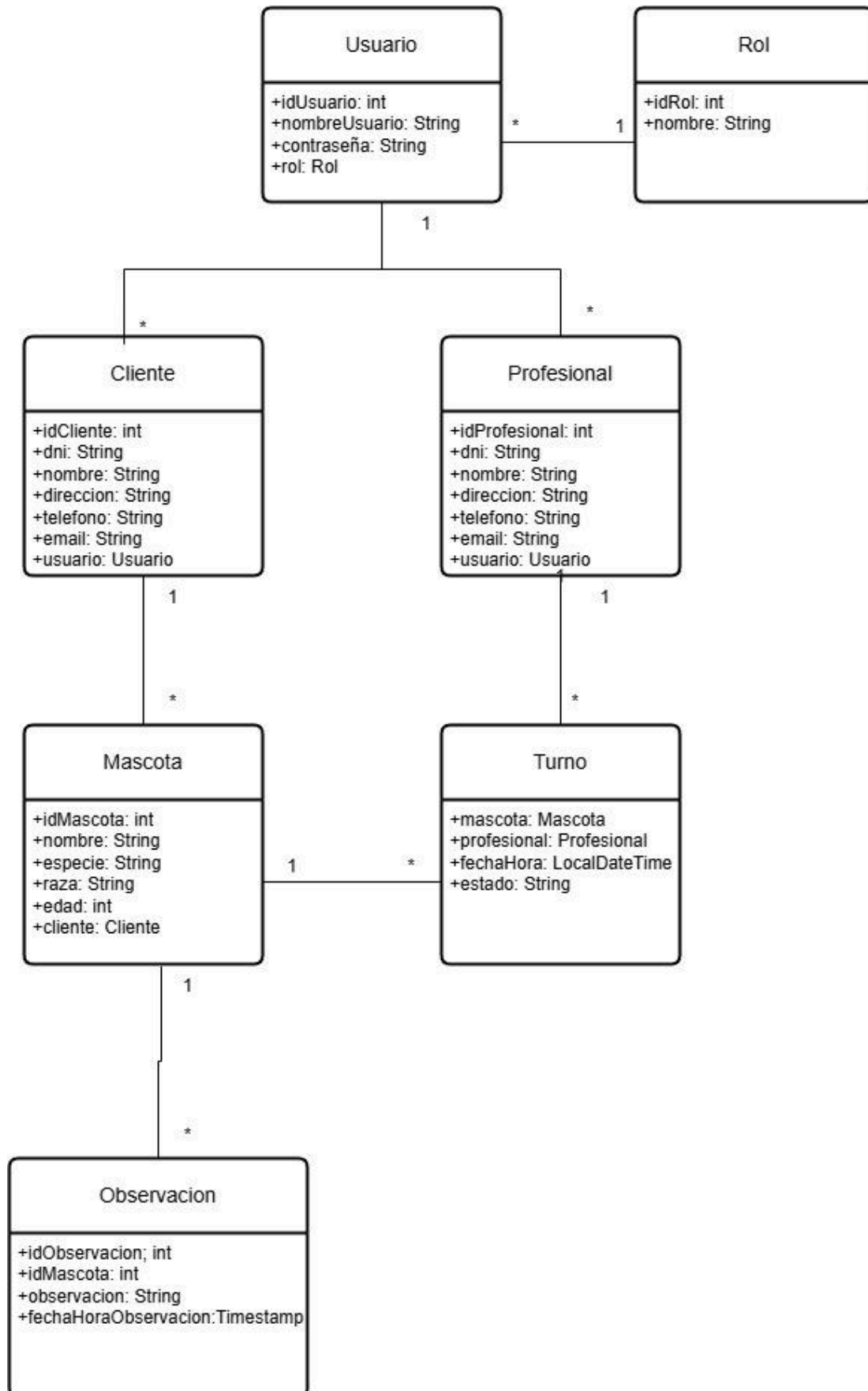
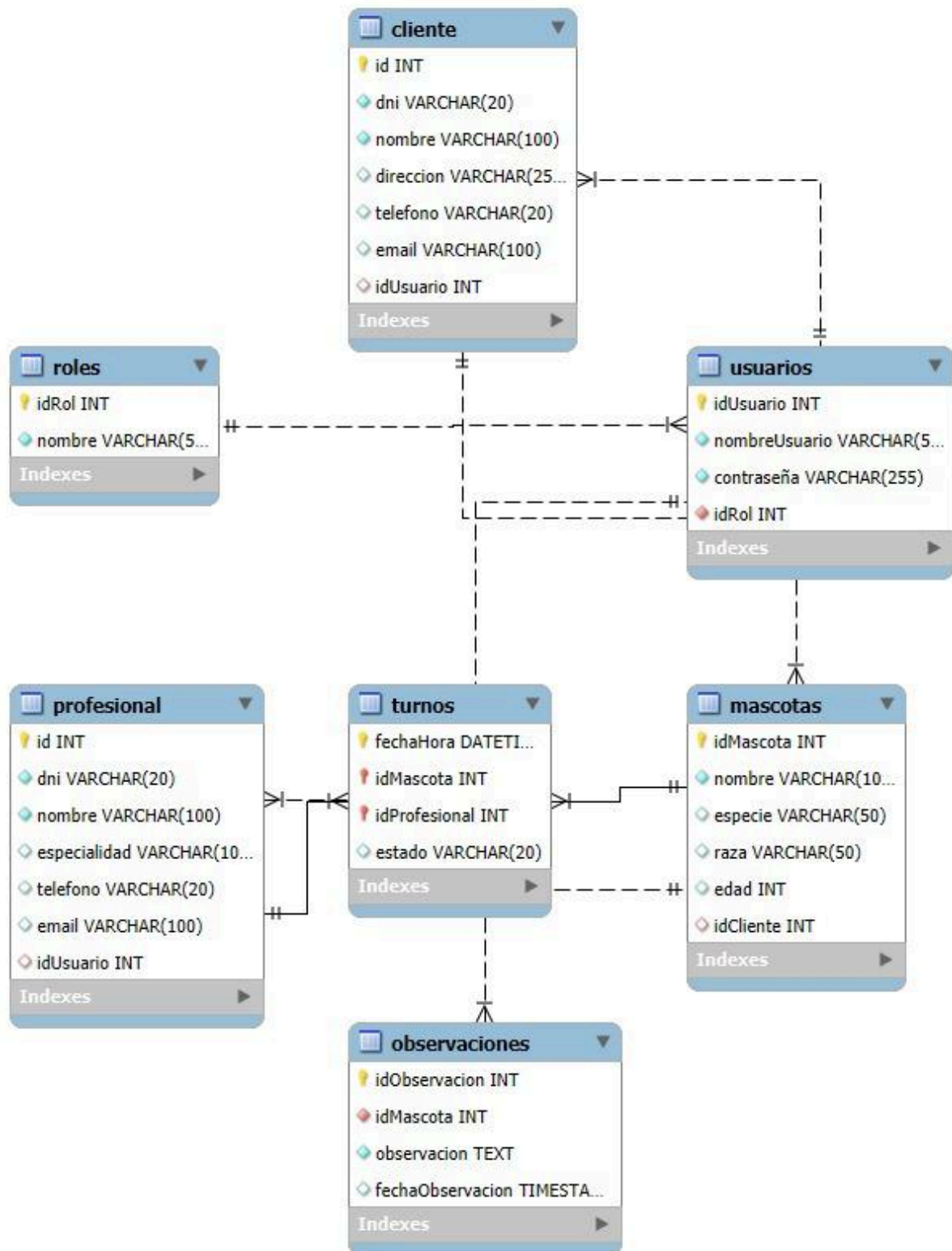


Diagrama de Datos

(Diagrama "Crow Foot")



Caso de Uso Resumen Reestructurado

CURS 01 - Realizar consulta

Nivel	Estructura	Alcance	Caja	Interacción	Instanciación
Resumen	Reestructurado	Sistema	Negra	Semántica	Real

Meta del caso de uso: Recibir atención profesional

ACTORES

Primario: Paciente

Otros: Profesional

PRECONDICIONES (de negocio): <vacío>

PRECONDICIONES (de sistema):

- Existen horarios disponibles del profesional
- Existen profesionales y pacientes registrados.
- Profesional se encuentra con sesión iniciada en el sistema durante las consultas.

DISPARADOR: Paciente selecciona mascota

FLUJO DE SUCESOS

CAMINO BÁSICO:

- 1- El paciente crea un turno para su mascota con un profesional a elección en un horario determinado invocando <CUU01 Pedir turno>.
- 2- A la hora y fecha del turno, el paciente es atendido por el profesional invocando <CUU02 Recepcionar turno>.

CAMINOS ALTERNATIVOS:

- 1.a <Posterior>: Paciente decide cancelar el turno:
 - 1.a.1 Fin CU.
- 2.a <Previo> Profesional cancela el turno:
 - 2.a.1 Fin CU.

POSTCONDICIONES (de negocio)

- **Éxito:** Consulta realizada en el turno asignado
- **Fracaso:** Consulta no realizada por cancelación
- **Éxito alternativo:** <vacío>

POSTCONDICIONES (de sistema)

- **Éxito:** Turno registrado como recepcionado.
- **Fracaso:** Turno registrado como cancelado.
- **Éxito alternativo:** <vacío>

Casos de Uso de Usuario

CUU 01 - Pedir turno

Dimensiones de clasificación

Nivel	Estructura	Alcance	Caja	Instalación	Interacción
Usuario	Sin estructurar	Sistema	Negra	Real	Semántica

Meta del usuario: Registrar turno para el paciente

ACTORES:

Primario: Paciente

Secundario: Profesional

PRECONDICIONES:

De negocio: <vacío>

De sistema:

- El paciente y el profesional deben estar registrados en el sistema
- El paciente debe tener una mascota registrada en el sistema

DISPARADOR: El usuario accede a la página "Seleccionar Turno".

Flujo Básico

CAMINO BÁSICO:

1. El sistema muestra un formulario para seleccionar un profesional disponible y elegir fecha y hora del turno.
2. El usuario selecciona un profesional y una fecha/hora disponible.
3. El usuario confirma la solicitud haciendo clic en el botón "Confirmar Turno".
4. El sistema procesa la solicitud y guarda el turno en la base de datos.
5. El sistema muestra un mensaje de confirmación al usuario.

CAMINOS ALTERNATIVOS:

2.a<posterior> El turno no está disponible:

2.a.1 El sistema informa al usuario y solicita una nueva selección.

2.a.2 Sigue en paso 2

5.a<posterior> El usuario decide cancelar su turno

5.a.1 El sistema registra la cancelación y abre ese turno otra vez. Fin de caso de uso

POSTCONDICIONES (de negocio)

Éxito: Paciente fue atendido

Fracaso : <vacío>

Éxito alternativo: El paciente canceló el turno

POSTCONDICIONES (de sistema)

Éxito: Turno registrado

Fracaso : <vacío>

Éxito alternativo: <vacío>

CUU 02 - Recepcionar turno

Dimensiones de clasificación

Nivel	Estructura	Alcance	Caja	Instalación	Interacción
Usuario	Sin estructurar	Sistema	Negra	Real	Semántica

Meta del usuario: Registrar la llegada del paciente a su turno médico

ACTORES:

Primario: Profesional

Secundario: Paciente

PRECONDICIONES:

De negocio: <vacío>

De sistema:

- El paciente y el profesional deben estar registrados en el sistema
- El paciente debe tener un turno con el profesional

DISPARADOR: El paciente pide un turno con el profesional

Flujo Básico

CAMINO BÁSICO:

1. El sistema muestra al profesional todos los turnos que realizó y debe realizar.
2. El profesional selecciona el turno del paciente.
3. El profesional confirma el turno haciendo clic en el botón "Recepcionar".
4. El sistema procesa la solicitud y guarda el estado del turno en la base de datos.
5. El sistema muestra un mensaje de confirmación al profesional.

CAMINOS ALTERNATIVOS:

3.a.<posterior> El profesional no puede realizar el turno:

3.a.1 El profesional realiza la cancelación haciendo clic en el botón "Cancelar".

3.a.2 El sistema registra la cancelación y abre ese turno otra vez. Fin de caso de uso

POSTCONDICIONES (de negocio)

Éxito: El profesional realiza el turno

Fracaso : <vacío>

Éxito alternativo: El profesional cancela el turno

POSTCONDICIONES (de sistema)

Éxito: Turno recepcionado

Fracaso : <vacío>

Éxito alternativo: <vacío>

Capturas de Pantallas

Login

Veterinaria

Login

Nombre de Usuario:

Ingrese su nombre de usuario

Contraseña:

Ingrese su contraseña

Iniciar Sesión

¿No tienes una cuenta? [Regístrate aquí.](#)

Veterinaria XYZ - Todos los derechos reservados

Contacto: 123-456-7890 | Email: contacto@veterinariaxyz.com

Error

Veterinaria

Página Principal

Ocurrió un Error

Detalles del error:

Volver al inicio

Veterinaria XYZ - Todos los derechos reservados

Contacto: 123-456-7890 | Email: contacto@veterinariaxyz.com

Pantallas de Cliente

Menu del Cliente

Veterinaria XYZ

[Inicio](#)[Información Personal](#)[Mis Mascotas](#)[Mis Turnos](#)[Ajustes](#)[Cerrar Sesión](#)

Bienvenido, Cliente

Selecciona una opción del menú para comenzar.

Veterinaria XYZ - Todos los derechos reservados

Contacto: 123-456-7890 | Email: contacto@veterinariaxyz.com

Registro de Cliente

Veterinaria

Registro

DNI:

Ingrese su DNI

Nombre:

Ingrese su nombre

Dirección:

Ingrese su dirección

Teléfono:

Ingrese su teléfono

Email:

Ingrese su email

Nombre de Usuario:

Ingrese su nombre de usuario

Contraseña:

Ingrese su contraseña

Registrarse

Veterinaria XYZ - Todos los derechos reservados

Registro Exitoso

Veterinaria

¡Registro Exitoso!

Tu cuenta ha sido creada correctamente. [Inicia sesión](#) para continuar.

Veterinaria XYZ - Todos los derechos reservados

Contacto: 123-456-7890 | Email: contacto@veterinariaxyz.com

Listado de Mascotas

Veterinaria

Página Principal

Mis Mascotas

Nombre	Especie	Raza	Edad	Acciones
Lola	Perro	Caniche	10	Editar Eliminar Sacar Turno

[Agregar Mascota](#) [Volver al Menú](#)

Veterinaria XYZ - Todos los derechos reservados
Contacto: 123-456-7890 | Email: contacto@veterinariaxyz.com

Agregar una Mascota

Veterinaria

Página Principal

Agregar Mascota

Nombre:

Ingrese el nombre de la mascota

Especie:

Ingrese la especie (por ejemplo, Perro, Gato)

Raza:

Ingrese la raza de la mascota

Edad:

Ingrese la edad de la mascota

Agregar Mascota

Información Personal del Cliente

Veterinaria

Página Principal

Información Personal

Datos Actuales

DNI	45656331
Nombre	Gero
Dirección	Brown 2873
Teléfono	3415697962
Email	geroypili2014@gmail.com

[Editar](#)

Veterinaria XYZ - Todos los derechos reservados
Contacto: 123-456-7890 | Email: contacto@veterinariaxyz.com

Editar Informacion Personal del Cliente

Veterinaria

Página Principal

Editar Información Personal

DNi:

45656331

Nombre:

Gero

Dirección:

Brown 2873

Teléfono:

3415697962

Email:

geroypli2014@gmail.com

Guardar Cambios

Veterinaria XYZ - Todos los derechos reservados
Contacto: 123-456-7890 | Email: contacto@veterinariaxyz.com

Editar Informacion de Login del Cliente

Veterinaria

Página Principal

Ajustes de Cuenta

Nuevo Nombre de Usuario:

Nueva Contraseña:

Guardar Cambios

[Volver al Menú](#)

Veterinaria XYZ - Todos los derechos reservados
Contacto: 123-456-7890 | Email: contacto@veterinariaxyz.com

Sacar Turno de Atención

Veterinaria

Página Principal

Sacar Turno

Seleccionar Profesional:

Carlos Maximo Chocobar - Veterinario

Fecha y Hora:

dd/mm/aaaa --:--

Confirmar Turno

[Volver](#)

Veterinaria XYZ - Todos los derechos reservados
Contacto: 123-456-7890 | Email: contacto@veterinariaxyz.com

Turnos del Cliente

Veterinaria

Página Principal

Mis Turnos

Mascota	Profesional	Fecha y Hora	Estado	Acciones
Lola	Carlos Maximo Chocobar	2025-04-03T20:21	Programado	Cancelar

[Volver al Menú](#)

Veterinaria XYZ - Todos los derechos reservados
Contacto: 123-456-7890 | Email: contacto@veterinariaxyz.com

Pantallas de Profesional

Menú del Profesional

Veterinaria XYZ

[Inicio](#) [Información Personal](#) [Próximos Turnos](#) [Información de Mascotas](#) [Ajustes](#) [Cerrar Sesión](#)

Bienvenido, Profesional

Selecciona una opción del menú para comenzar.

Veterinaria XYZ - Todos los derechos reservados
Contacto: 123-456-7890 | Email: contacto@veterinariaxyz.com

Información Personal del Profesional

Veterinaria

Página Principal

Información Personal del Profesional

Datos Actuales

DNI	46495922
Nombre	Carlos Maximo Chocobar
Teléfono	3416460470
Email	franmasche@gmail.com
Especialidad	Veterinario

[Editar](#)

Veterinaria XYZ - Todos los derechos reservados
Contacto: 123-456-7890 | Email: contacto@veterinariaxyz.com

Editar Informacion Personal del Cliente

Veterinaria

[Página Principal](#)

Editar Profesional

DNI:

46493922

Nombre:

Carlos Máximo Chocobar

Especialidad:

Veterinario

Teléfono:

3416460470

Email:

fransasche@gmail.com

Nombre de Usuario:

Ingrese el nombre de usuario

Contraseña:

Ingrese la contraseña

Guardar Profesional

Veterinaria XYZ - Todos los derechos reservados

Editar Informacion de Login del Cliente

Veterinaria

[Página Principal](#)

Ajustes de Cuenta

Nuevo Nombre de Usuario:

Nueva Contraseña:

Guardar Cambios

[Volver al Menú](#)

Veterinaria XYZ - Todos los derechos reservados

Ingresar Observación de Consulta

Veterinaria

[Página Principal](#)

Ingresar Observación

Observación:

Guardar Observación

[Cancelar](#)

Veterinaria XYZ - Todos los derechos reservados

Contacto: 123-456-7890 | Email: contacto@veterinariaxyz.com

Historial de Turnos del Profesional

Historial de Turnos

Fecha y Hora	Estado
2025-03-29T17:18	Cancelado
2025-03-28T07:00	Cancelado
2025-03-20T20:00	Cancelado
2025-03-20T16:00	Cancelado
2025-03-15T19:14	Cancelado
2025-03-08T17:30	Recepcionado

Observaciones de Mascota Atendida

Observaciones de la Mascota

Fecha	Observación
2025-03-30 18:57:03.0	Vómitos y diarrea
2025-03-08 15:57:49.0	Tos y estornudos frecuentes

[Volver](#)

Lista de Mascotas Atendidas

Mascotas Atendidas

Nombre	Especie	Raza	Edad	Acciones
Jack	Perro	Collie	12	Ver Historial Ver Observaciones
Gordon	Perro	Pug	9	Ver Historial Ver Observaciones

Listado de Turnos del Profesional

Veterinaria

Página Principal

Turnos del Profesional

Estado:

Todos

Ordenar por fecha:

Ascendente

Filtrar

Fecha y Hora	Mascota	Estado	Acciones
8 de marzo del 2025 a las 17:30	Jack	Recepcionado	Recepcionar Cancelar

Pantallas de Administrador

Menú del Administrador

Veterinaria XYZ

Inicio

Gestión de Clientes

Gestión de Profesionales

Gestión de Turnos

Ajustes

Cerrar Sesión

Bienvenido, Admin

Selecciona una opción del menú para comenzar.

Veterinaria XYZ - Todos los derechos reservados

Contacto: 123-456-7890 | Email: contacto@veterinariaxyz.com

Turnos Actuales

Veterinaria

Página Principal

Turnos Actuales

Mascota	Profesional	Fecha y Hora	Estado	Acciones
Jack	Carlos Maximo Chocobar	2025-03-08T17:30	Recepcionado	Recepcionar Cancelar
Jack	Carlos Maximo Chocobar	2025-03-15T19:14	Cancelado	Recepcionar Cancelar
				Recepcionar

Editar Perfil de Cliente

Veterinaria

Página Principal

Editar Cliente

DNI:

2347689

Nombre:

Juan Perez

Dirección:

Entre Rios 1400

Teléfono:

3415151448

Email:

ejemplo@ejemplo.mx

Nombre de Usuario:

Ingrese el nombre de usuario

Contraseña:

Ingrese la contraseña

Guardar Cliente

Veterinaria XYZ - Todos los derechos reservados

Registrar Nuevo Profesional

Veterinaria

Página Principal

Agregar Nuevo Profesional

DNI:

Ingrese el DNI

Nombre:

Ingrese el nombre

Especialidad:

Ingrese la especialidad

Teléfono:

Ingrese el teléfono

Email:

Ingrese el email

Nombre de Usuario:

Ingrese el nombre de usuario

Contraseña:

Ingrese la contraseña

Guardar Profesional

Veterinaria XYZ - Todos los derechos reservados

Listado de Todas las Mascotas

Veterinaria

Página Principal

Lista de Mascotas

ID	Nombre	Especie	Raza	Edad	Acciones
7	Jack	Perro	Collie	12	Editar Eliminar
8	Gordon	Perro	Pug	9	Editar Eliminar

Agregar Nueva Mascota

Veterinaria XYZ - Todos los derechos reservados
Contacto: 123-456-7890 | Email: contacto@veterinariaxyz.com

Editar una Mascota

Veterinaria

Página Principal

Editar Mascota

Nombre:

Jack

Especie:

Perro

Raza:

Collie

Edad:

12

Cliente:

Facundo Munne

Actualizar

Listado de Todos los Clientes

Veterinaria

Página Principal

Lista de Clientes

ID	DNI	Nombre	Dirección	Teléfono	Email	Acciones
5	44523096	Facundo Munne	Buenos Aires 1430	3416470473	fmunne@gmail.com	Editar Eliminar Ver Mascotas
8	2347689	Juan Perez	Entre Rios 1400	3415151448	ejemplo@ejemplo.mx	Editar Eliminar Ver Mascotas

[Agregar Nuevo Cliente](#)

Veterinaria XYZ - Todos los derechos reservados

Contacto: 123-456-7890 | Email: contacto@veterinariaxyz.com

Listado de Todos los Profesionales

Veterinaria

Página Principal

Lista de Profesionales

ID	DNI	Nombre	Especialidad	Teléfono	Email	Acciones
2	46495922	Carlos Maximo Chocobar	Veterinario	3416460470	franmasche@gmail.com	Editar Eliminar Ver Turnos

[Agregar Nuevo Profesional](#)

Veterinaria XYZ - Todos los derechos reservados

Contacto: 123-456-7890 | Email: contacto@veterinariaxyz.com

Listado de Todos los Turnos de los Profesionales

Veterinaria

[Página Principal](#)

Turnos del Profesional

ID Mascota	Nombre Mascota	Nombre Profesional	Fecha y Hora	Estado
7	Jack	Carlos Maximo Chocobar	2025-03-08T17:30	Recepcionado
7	Jack	Carlos Maximo Chocobar	2025-03-15T19:14	Cancelado
7	Jack	Carlos Maximo Chocobar	2025-03-20T16:00	Cancelado
7	Jack	Carlos Maximo Chocobar	2025-03-20T20:00	Cancelado
8	Gordon	Carlos Maximo Chocobar	2025-03-21T15:00	Recepcionado
7	Jack	Carlos Maximo Chocobar	2025-03-28T07:00	Cancelado
7	Jack	Carlos Maximo Chocobar	2025-03-29T17:18	Cancelado
8	Gordon	Carlos Maximo Chocobar	2025-03-31T10:00	Recepcionado
8	Gordon	Carlos Maximo Chocobar	2025-04-03T08:00	Programado

[Volver a la lista de profesionales](#)

Fragmento deCodigo

C.U. elegido: CUU 01 - Pedir turno

CU elegido: Pedir Turno

misMascotas.jsp

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Mis Mascotas</title>
  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/@picocss/pico@latest/css/pico.min.css">
</head>
<body>
  <%@ include file="../public/header.jsp" %>
  <header class="container">
    <h1>Mis Mascotas</h1>
  </header>
  <main class="container">
    <!-- Verificar si el usuario está autenticado -->
    <c:if test="${empty sessionScope.usuario}">
      <p>No estás autenticado. Por favor, <a
href="${pageContext.request.contextPath}/public/login.jsp">inicia sesión</a>.</p>
    </c:if>
    <c:if test="${not empty sessionScope.usuario}">
      <!-- Mostrar mensaje si no hay mascotas -->
      <c:if test="${empty mascotas}">
        <p>No tienes mascotas registradas.</p>
      </c:if>
    </c:if>
  </main>
</body>
</html>
```

```

<!-- Mostrar la tabla de mascotas -->
<c:if test="${not empty mascotas}">
    <table>
        <thead>
            <tr>
                <th>Nombre</th>
                <th>Especie</th>
                <th>Raza</th>
                <th>Edad</th>
                <th>Acciones</th>
            </tr>
        </thead>
        <tbody>
            <c:forEach var="mascota" items="${mascotas}">
                <tr>
                    <td>${mascota.nombre}</td>
                    <td>${mascota.especie}</td>
                    <td>${mascota.raza}</td>
                    <td>${mascota.edad}</td>
                    <td>
                        <!-- Enlaces para editar, eliminar y sacar turno -->
                        <a
href="crudmascota?action=update&id=${mascota.idMascota}&clienteId=${sessionScope.idCliente}"
class="secondary">Editar</a>
                        <a
href="crudmascota?action=delete&id=${mascota.idMascota}&clienteId=${sessionScope.idCliente}"
class="contrast">Eliminar</a>
                        <a
href="${pageContext.request.contextPath}/sacarTurno?idMascota=${mascota.idMascota}"
class="button">Sacar Turno</a>
                    </td>
                </tr>
            </c:forEach>
        </tbody>
    </table>
</c:if>
<!-- Enlace para agregar una nueva mascota -->
<a
href="${pageContext.request.contextPath}/cliente/agregarMascota.jsp?clienteId=${sessionScope.idCl
iente}" class="button">Agregar Mascota</a>
<!-- Enlace para volver al menú -->
<a href="${pageContext.request.contextPath}/public/menu.jsp" class="button">Volver al
Menú</a>
</c:if>
</main>
<%@ include file="../public/footer.jsp" %>
</body>
</html>

```

SacarTurnoServlet.java

```
@WebServlet("/sacarTurno")
public class SacarTurnoServlet extends HttpServlet {
    private DataProfesional dataProfesional = new DataProfesional();
    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {
        try {
            // Obtener el idMascota de la solicitud
            String idMascotaParam = request.getParameter("idMascota");
            if (idMascotaParam == null || idMascotaParam.isEmpty()) {
                // Si no se recibe el idMascota, redirigir a la lista de mascotas
                response.sendRedirect(request.getContextPath() + "/listarMascotas");
                return;
            }
            // Convertir el idMascota a entero
            int idMascota = Integer.parseInt(idMascotaParam);
            // Obtener la lista de profesionales disponibles
            List<Profesional> profesionales = dataProfesional.getAll();
            // Guardar el idMascota y la lista de profesionales en el alcance de la solicitud
            request.setAttribute("idMascota", idMascota);
            request.setAttribute("profesionales", profesionales);
            // Redirigir al JSP para seleccionar profesional y horario
            request.getRequestDispatcher("/cliente/seleccionarTurno.jsp").forward(request, response);
        } catch (NumberFormatException e) {
            // Capturar el error si el idMascota no es un número válido
            System.out.println("ERROR: El idMascota no es un número válido.");
            request.setAttribute("error", "El ID de la mascota no es válido.");
            request.getRequestDispatcher("/public/error.jsp").forward(request, response);
        } catch (Exception e) {
            // Capturar cualquier otro error inesperado
            System.out.println("ERROR INESPERADO: " + e.getMessage());
            e.printStackTrace();
            request.setAttribute("error", "Ocurrió un error inesperado al intentar sacar el turno. Intente nuevamente.");
            request.getRequestDispatcher("/public/error.jsp").forward(request, response);
        }
    }
}
```

DataProfesional.java

```
public class DataProfesional {

    public Profesional getById(int id) {
        String query = "SELECT p.*, u.idUsuario, u.nombreUsuario, u.contraseña, r.idRol, r.nombre
AS nombreRol " +
            "FROM Profesional p " +
            "JOIN Usuarios u ON p.idUsuario = u.idUsuario " +
            "JOIN Roles r ON u.idRol = r.idRol " +
            "WHERE p.id = ?";
```

```

Profesional profesional = null;
Connection conn = null;
PreparedStatement stmt = null;
ResultSet rs = null;

```

```

try {
    conn = DbConnector.getInstancia().getConn();
    stmt = conn.prepareStatement(query);
    stmt.setInt(1, id);
    rs = stmt.executeQuery();

    if (rs.next()) {
        // Crear el objeto Usuario
        Usuario usuario = new Usuario();
        usuario.setIdUsuario(rs.getInt("idUsuario"));
        usuario.setNombreUsuario(rs.getString("nombreUsuario"));
        usuario.setContraseña(rs.getString("contraseña"));

        // Crear el objeto Rol
        Rol rol = new Rol();
        rol.setIdRol(rs.getInt("idRol"));
        rol.setNombre(rs.getString("nombreRol"));

        // Asignar el Rol al Usuario
        usuario.setRol(rol);
        // Crear el objeto Profesional
        profesional = new Profesional();
        profesional.setIdProfesional(rs.getInt("id"));
        profesional.setUsuario(usuario);
        profesional.setDni(rs.getString("dni"));
        profesional.setNombre(rs.getString("nombre"));
        profesional.setEspecialidad(rs.getString("especialidad"));
        profesional.setTelefono(rs.getString("telefono"));
        profesional.setEmail(rs.getString("email"));
    }
} catch (SQLException e) {
    e.printStackTrace();
} finally {
    try {
        if (rs != null) rs.close();
        if (stmt != null) stmt.close();
        if (conn != null) DbConnector.getInstancia().releaseConn();
    } catch (SQLException e) {
        e.printStackTrace();
    }
}
return profesional;
}

```

```

public List<Profesional> getAll() {
    List<Profesional> profesionales = new ArrayList<>();

```

```
String query = "SELECT p.*, u.idUsuario, u.nombreUsuario, u.contraseña, r.idRol, r.nombre  
AS nombreRol " +
```

```
"FROM Profesional p " +
```

```
"JOIN Usuarios u ON p.idUsuario = u.idUsuario " +
```

```
"JOIN Roles r ON u.idRol = r.idRol";
```

```
Connection conn = null;
```

```
PreparedStatement stmt = null;
```

```
ResultSet rs = null;
```

```
try {
```

```
    conn = DbConnector.getInstancia().getConn();
```

```
    stmt = conn.prepareStatement(query);
```

```
    rs = stmt.executeQuery();
```

```
while (rs.next()) {
```

```
    // Crear el objeto Usuario
```

```
    Usuario usuario = new Usuario();
```

```
    usuario.setIdUsuario(rs.getInt("idUsuario"));
```

```
    usuario.setNombreUsuario(rs.getString("nombreUsuario"));
```

```
    usuario.setContraseña(rs.getString("contraseña"));
```

```
    // Crear el objeto Rol
```

```
    Rol rol = new Rol();
```

```
    rol.setIdRol(rs.getInt("idRol"));
```

```
    rol.setNombre(rs.getString("nombreRol"));
```

```
    // Asignar el Rol al Usuario
```

```
    usuario.setRol(rol);
```

```
    // Crear el objeto Profesional
```

```
    Profesional profesional = new Profesional();
```

```
    profesional.setIdProfesional(rs.getInt("id"));
```

```
    profesional.setUsuario(usuario);
```

```
    profesional.setDni(rs.getString("dni"));
```

```
    profesional.setNombre(rs.getString("nombre"));
```

```
    profesional.setEspecialidad(rs.getString("especialidad"));
```

```
    profesional.setTelefono(rs.getString("telefono"));
```

```
    profesional.setEmail(rs.getString("email"));
```

```
    // Agregar el profesional a la lista
```

```
    profesionales.add(profesional);
```

```
}
```

```
} catch (SQLException e) {
```

```
    e.printStackTrace();
```

```
} finally {
```

```
    try {
```

```
        if (rs != null) rs.close();
```

```
        if (stmt != null) stmt.close();
```

```
        if (conn != null) DbConnector.getInstancia().releaseConn();
```

```
    } catch (SQLException e) {
```

```
        e.printStackTrace();
```

```
    }
```

```
}
```

```
return profesionales;
```

```
}  
}
```

Profesional.java

```
public class Profesional {  
    private int idProfesional;  
    private Usuario usuario;  
    private String dni;  
    private String nombre;  
    private String especialidad;  
    private String telefono;  
    private String email;  
  
    public Profesional(int id,Usuario usuario,String dni, String nombre, String especialidad, String  
telefono, String email) {  
        this.idProfesional = id;  
        this.usuario = usuario;  
        this.dni=dni;  
        this.nombre = nombre;  
        this.especialidad = especialidad;  
        this.telefono = telefono;  
        this.email = email;  
    }  
  
    public Profesional(int id,String dni, String nombre, String especialidad, String telefono, String  
email) {  
        this.idProfesional = id;  
        this.dni=dni;  
        this.nombre = nombre;  
        this.especialidad = especialidad;  
        this.telefono = telefono;  
        this.email = email;  
    }  
  
    public Profesional() {  
  
    }  
  
    public int getIdProfesional() {  
        return idProfesional;  
    }  
    public void setIdProfesional(int idProfesional) {  
        this.idProfesional = idProfesional;  
    }  
  
    public Usuario getUsuario() {  
        return usuario;  
    }  
    public void setUsuario(Usuario usuario) {  
        this.usuario = usuario;  
    }  
}
```

```

    }

    public String getDni() {
        return dni;
    }
    public void setDni(String dni) {
        this.dni = dni;
    }
    public String getNombre() {
        return nombre;
    }
    public void setNombre(String nombre) {
        this.nombre = nombre;
    }
    public String getEspecialidad() {
        return especialidad;
    }
    public void setEspecialidad(String especialidad) {
        this.especialidad = especialidad;
    }
    public String getTelefono() {
        return telefono;
    }
    public void setTelefono(String telefono) {
        this.telefono = telefono;
    }
    public String getEmail() {
        return email;
    }
    public void setEmail(String email) {
        this.email = email;
    }
}
}

```

seleccionarTurno.jsp

```

<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Sacar Turno</title>
    <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/@picocss/pico@latest/css/pico.min.css">
</head>
<body>
    <%@ include file="../../public/header.jsp" %>
    <header class="container">
        <h1>Sacar Turno</h1>
    </header>
    <main class="container">
        <form action="${pageContext.request.contextPath}/guardarTurno" method="post">

```



```

<input type="hidden" name="idMascota" value="{idMascota}">
<label for="profesional">Seleccionar Profesional:</label>
<select id="profesional" name="idProfesional" required>
    <c:forEach var="profesional" items="{profesionales}">
        <option value="{profesional.idProfesional}">{profesional.nombre} -
        {profesional.especialidad}</option>
    </c:forEach>
</select><br><br>
<label for="fechaHora">Fecha y Hora:</label>
<input type="datetime-local" id="fechaHora" name="fechaHora" required><br><br>
<button type="submit">Confirmar Turno</button>
</form>
<a href="{pageContext.request.contextPath}/listarMascotas" class="button">Volver</a>
</main>
<%@ include file="../public/footer.jsp" %>
</body>
</html>

```

GuardarTurnoServlet.java

```

@WebServlet("/guardarTurno")
public class GuardarTurnoServlet extends HttpServlet {
    private DataTurno dataTurno;
    private DataMascota dataMascota;
    private DataProfesional dataProfesional;
    @Override
    public void init() throws ServletException {
        super.init();
        dataTurno = new DataTurno();
        dataMascota = new DataMascota();
        dataProfesional = new DataProfesional();
    }
    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        HttpSession session = request.getSession(false);
        String redirectUrl = request.getContextPath() + "/listarMascotas";

        int idMascota = 0; // Declarar idMascota fuera del try, para que sea accesible en los catch
        try {
            // 1. Validar sesión
            if (session == null || session.getAttribute("usuario") == null) {
                response.sendRedirect(request.getContextPath() + "/public/login.jsp");
                return;
            }
            // 2. Validar y parsear parámetros
            idMascota = Integer.parseInt(request.getParameter("idMascota"));
            int idProfesional = Integer.parseInt(request.getParameter("idProfesional"));
            LocalDateTime fechaHora = LocalDateTime.parse(
                request.getParameter("fechaHora"),

```

```

        DateTimeFormatter.ISO_LOCAL_DATE_TIME
    );
    // 3. Validar fecha futura
    if (fechaHora.isBefore(LocalDateTime.now())) {
        throw new IllegalArgumentException("No se pueden agendar turnos en fechas pasadas");
    }
    // 4. Obtener mascota y profesional
    Mascota mascota = dataMascota.getByld(idMascota);
    Profesional profesional = dataProfesional.getByld(idProfesional);
    if (mascota == null || profesional == null) {
        throw new IllegalArgumentException("Mascota o profesional no encontrados");
    }
    // 5. Verificar disponibilidad del profesional
    if (!dataTurno.isProfesionalAvailable(idProfesional, fechaHora)) {
        throw new IllegalStateException("El profesional no está disponible en ese horario");
    }
    // 6. Crear y guardar turno
    Turno turno = new Turno();
    turno.setMascota(mascota);
    turno.setProfesional(profesional);
    turno.setFechaHora(fechaHora);
    turno.setEstado("Programado");

    dataTurno.add(turno);
    // 7. Mensaje de éxito
    session.setAttribute("successMessage", "Turno agendado exitosamente para " +
        fechaHora.format(DateTimeFormatter.ofPattern("dd/MM/yyyy HH:mm")));

} catch (NumberFormatException e) {
    // Redirigir a error.jsp con detalles del error
    request.setAttribute("errorType", "ID inválido");
    request.setAttribute("errorMessage", "Los IDs deben ser números válidos");
    request.setAttribute("errorRedirect", request.getContextPath() + "/sacarTurno?idMascota=" +
idMascota);
    request.setAttribute("errorButtonText", "Volver");
    request.setAttribute("errorDebug", e.getMessage());
    request.getRequestDispatcher("/public/error.jsp").forward(request, response);
    return;
} catch (DateTimeParseException e) {
    request.setAttribute("errorType", "Fecha inválida");
    request.setAttribute("errorMessage", "Formato de fecha/hora inválido (use:
yyyy-MM-ddTHH:mm)");
    request.setAttribute("errorRedirect", request.getContextPath() + "/sacarTurno?idMascota=" +
idMascota);
    request.setAttribute("errorButtonText", "Volver");
    request.setAttribute("errorDebug", e.getMessage());
    request.getRequestDispatcher("/public/error.jsp").forward(request, response);
    return;
} catch (IllegalArgumentException e) {
    request.setAttribute("errorType", "Entrada inválida");
    request.setAttribute("errorMessage", e.getMessage());

```

```

        request.setAttribute("errorRedirect", request.getContextPath() + "/sacarTurno?idMascota=" +
idMascota);
        request.setAttribute("errorButtonText", "Volver");
        request.setAttribute("errorDebug", e.getMessage());
        request.getRequestDispatcher("/public/error.jsp").forward(request, response);
        return;
    } catch (IllegalStateException e) {
        request.setAttribute("errorType", "Disponibilidad del profesional");
        request.setAttribute("errorMessage", e.getMessage());
        request.setAttribute("errorRedirect", request.getContextPath() + "/sacarTurno?idMascota=" +
idMascota);
        request.setAttribute("errorButtonText", "Volver");
        request.setAttribute("errorDebug", e.getMessage());
        request.getRequestDispatcher("/public/error.jsp").forward(request, response);
        return;
    } catch (Exception e) {
        request.setAttribute("errorType", "Error general");
        request.setAttribute("errorMessage", "Error al agendar el turno");
        request.setAttribute("errorRedirect", request.getContextPath() + "/sacarTurno?idMascota=" +
idMascota);
        request.setAttribute("errorButtonText", "Volver");
        request.setAttribute("errorDebug", e.getMessage());
        request.getRequestDispatcher("/public/error.jsp").forward(request, response);
        return;
    }
}
response.sendRedirect(redirectUrl);
}
}

```

DataMascota.java

```

public class DataMascota {
    public Mascota getByld(int id) {
        String query = "SELECT * FROM Mascotas WHERE idMascota = ?";
        Mascota mascota = null;
        Connection conn = null;
        PreparedStatement stmt = null;
        ResultSet rs = null;

        try {
            conn = DbConnector.getInstancia().getConn();
            stmt = conn.prepareStatement(query);
            stmt.setInt(1, id);
            rs = stmt.executeQuery();

            if (rs.next()) {
                DataCliente dataCliente = new DataCliente();
                Cliente cliente = dataCliente.getByld(rs.getInt("idCliente"));

                mascota = new Mascota(
                    rs.getInt("idMascota"),
                    rs.getString("nombre"),

```

```

        rs.getString("especie"),
        rs.getString("raza"),
        rs.getInt("edad"),
        cliente
    );
}
} catch (SQLException e) {
    e.printStackTrace();
} finally {
    try {
        if (rs != null) rs.close();
        if (stmt != null) stmt.close();
        if (conn != null) DbConnector.getInstance().releaseConn();
    } catch (SQLException e) {
        e.printStackTrace();
    }
}
}
return mascota;
}

public List<Mascota> getByClientId(int clientId) {
    List<Mascota> mascotas = new ArrayList<>();
    String query = "SELECT * FROM Mascotas WHERE idCliente = ?";
    Connection conn = null;
    PreparedStatement stmt = null;
    ResultSet rs = null;
    try {
        conn = DbConnector.getInstance().getConn();
        stmt = conn.prepareStatement(query);
        stmt.setInt(1, clientId); // Usamos el clientId directamente
        rs = stmt.executeQuery();
        while (rs.next()) {
            // No es necesario obtener el cliente de nuevo, ya que lo tenemos en el
            Mascota mascota = new Mascota(
                rs.getInt("idMascota"),
                rs.getString("nombre"),
                rs.getString("especie"),
                rs.getString("raza"),
                rs.getInt("edad"),
                new Cliente(clientId, null, null, null, null, null, null) // Pasamos el cliente
                directamente o con los datos disponibles
            );
            mascotas.add(mascota);
        }
    } catch (SQLException e) {
        e.printStackTrace();
    } finally {
        try {
            if (rs != null) rs.close();
            if (stmt != null) stmt.close();
            if (conn != null) DbConnector.getInstance().releaseConn();
        } catch (SQLException e) {

```

servlet

```

        e.printStackTrace();
    }
}
return mascotas;
}
}

```

Mascota.java

```

public class Mascota {
    private int idMascota;
    private String nombre;
    private String especie;
    private String raza;
    private int edad;
    private Cliente cliente;

    public Mascota(String nombre, String especie, String raza, int edad, Cliente cliente) {
        this.nombre = nombre;
        this.especie = especie;
        this.raza = raza;
        this.edad = edad;
        this.cliente = cliente;
    }

    public Mascota(int idMascota, String nombre, String especie, String raza, int edad,
Cliente cliente) {
        this.idMascota = idMascota;
        this.nombre = nombre;
        this.especie = especie;
        this.raza = raza;
        this.edad = edad;
        this.cliente = cliente;
    }

    public Mascota() {
        // TODO Auto-generated constructor stub
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public String getEspecie() {
        return especie;
    }

    public void setEspecie(String especie) {
        this.especie = especie;
    }
}

```

```

    }
    public String getRaza() {
        return raza;
    }
    public void setRaza(String raza) {
        this.raza = raza;
    }
    public int getEdad() {
        return edad;
    }
    public void setEdad(int edad) {
        this.edad = edad;
    }
    public int getIdMascota() {
        return idMascota;
    }
    public void setIdMascota(int idMascota) {
        this.idMascota = idMascota;
    }
    public Cliente getCliente() {
        return cliente;
    }
    public void setCliente(Cliente cliente) {
        this.cliente = cliente;
    }
}

```

DataTurno.java

```

public class DataTurno {

    public void add(Turno turno) {
        String checkQuery = "SELECT COUNT(*) FROM Turnos WHERE idProfesional = ?
AND fechaHora = ? AND estado = 'ACTIVO'";
        String insertQuery = "INSERT INTO Turnos (fechaHora, idMascota, idProfesional,
estado) VALUES (?, ?, ?, ?)";
        Connection conn = null;
        PreparedStatement stmt = null;
        ResultSet rs = null;
        try {
            conn = DbConnector.getInstance().getConn();

            // Desactivar autocommit
            conn.setAutoCommit(false);
            // Verificar si ya existe un turno para el profesional en el mismo horario
            stmt = conn.prepareStatement(checkQuery);
            stmt.setInt(1, turno.getProfesional().getIdProfesional());
            stmt.setTimestamp(2, Timestamp.valueOf(turno.getFechaHora())); // Fecha y
hora de inicio del turno

```

```

rs = stmt.executeQuery();
if (rs.next() && rs.getInt(1) > 0) {
    // Si ya existe un turno en esa fechaHora, lanzar excepción
    throw new SQLException("Ya existe un turno para este profesional en el
horario especificado.");
}
// Si no existe un turno con la misma fechaHora, se inserta el nuevo turno
stmt = conn.prepareStatement(insertQuery);
stmt.setTimestamp(1, Timestamp.valueOf(turno.getFechaHora()));
stmt.setInt(2, turno.getMascota().getIdMascota());
stmt.setInt(3, turno.getProfesional().getIdProfesional());
stmt.setString(4, turno.getEstado());
int rowsInserted = stmt.executeUpdate();
if (rowsInserted > 0) {
    System.out.println("El turno se guardó correctamente.");
}
// Hacer commit explícito de la transacción
conn.commit();
} catch (SQLException e) {
    e.printStackTrace();
    try {
        if (conn != null) {
            conn.rollback(); // Hacer rollback en caso de error
        }
    } catch (SQLException rollbackEx) {
        rollbackEx.printStackTrace();
    }
    throw new RuntimeException("Error al agregar el turno: " + e.getMessage()); //
O lanzar una excepción personalizada
} finally {
    try {
        if (rs != null) rs.close();
        if (stmt != null) stmt.close();
        if (conn != null) {
            conn.setAutoCommit(true); // Restablecer autocommit a true
            DbConnector.getInstance().releaseConn();
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
}
}

public boolean isProfesionalAvailable(int idProfesional, LocalDateTime fechaHora) {
    String query = "SELECT COUNT(*) FROM Turnos WHERE idProfesional = ? AND
fechaHora = ?";
    Connection conn = null;
    PreparedStatement stmt = null;
    ResultSet rs = null;
    try {
        conn = DbConnector.getInstance().getConn();
        stmt = conn.prepareStatement(query);
        stmt.setInt(1, idProfesional);

```

```

        stmt.setTimestamp(2, java.sql.Timestamp.valueOf(fechaHora));
        rs = stmt.executeQuery();
        if (rs.next()) {
            int count = rs.getInt(1);
            return count == 0;
        }
    } catch (SQLException e) {
        e.printStackTrace();
    } finally {
        try {
            if (rs != null) rs.close();
            if (stmt != null) stmt.close();
            if (conn != null) DbConnector.getInstance().releaseConn();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
    return false;
}
}

```

Turno.java

```

public class Turno {

    private Mascota mascota;
    private Profesional profesional;
    private LocalDateTime fechaHora;
    private String estado;

    public Turno(Mascota mascota, Profesional profesional, LocalDateTime fechaHora,
String estado) {

        this.mascota = mascota;
        this.profesional = profesional;
        this.fechaHora = fechaHora;
        this.estado= estado;
    }
    public String getEstado() {
        return estado;
    }
    public void setEstado(String estado) {
        this.estado = estado;
    }
    public Turno() {
        // TODO Auto-generated constructor stub
    }
    public Mascota getMascota() {

```



```
        return mascota;
    }
    public void setMascota(Mascota mascota) {
        this.mascota = mascota;
    }
    public Profesional getProfesional() {
        return profesional;
    }
    public void setProfesional(Profesional profesional) {
        this.profesional = profesional;
    }
    public LocalDateTime getFechaHora() {
        return fechaHora;
    }
    public void setFechaHora(LocalDateTime fechaHora) {
        this.fechaHora = fechaHora;
    }
}

}
```